Project AETHER: Architectural Evolution for Thermochemical Hypersonic Environment Reconstruction

Executive Summary

The validation of novel material systems for hypersonic flight represents one of the most computationally demanding frontiers in modern aerospace engineering. As vehicles traverse near-space environments at speeds exceeding Mach 5, they encounter extreme thermochemical conditions, including plasma sheath formation, surface ablation, and non-equilibrium gas dynamics. Previous efforts, such as the foundational research entitled "Understanding Near-Space Weather Encounters of Hypersonic Flight Novel Material Systems", laid the groundwork for characterizing these interactions. However, legacy computational fluid dynamics (CFD) and Direct Simulation Monte Carlo (DSMC) methods struggle to scale on modern hardware, creating a bottleneck in material discovery and real-time flight envelope prediction.

This report proposes Project AETHER, a complete architectural overhaul of the legacy research code. By integrating state-of-the-art methodologies from high-performance computing (HPC) and artificial intelligence (AI), Project AETHER aims to create a differentiable, GPU-accelerated, and AI-augmented simulation framework. The project leverages JAX and OpenAI Triton for custom kernel generation, Physics-Informed Transformers for surrogate modeling, and Disaggregated Serving architectures for real-time digital twinning. This unified approach not only addresses the multi-scale physics of ablation and plasma formation but also establishes a highly optimized pipeline leveraging low-level GPU performance engineering, distributed system design, and scientific machine learning (SciML).

1 Part I: Theoretical Foundations and Computational Challenges

1.1 The Hypersonic Environment: Near-Space Weather and Plasma Sheaths

The primary physical domain of Project AETHER is the hypersonic flight corridor, where the interaction between the vehicle and the rarefied atmosphere generates a shock layer of ionized gas, or plasma. This environment is characterized by extreme enthalpy, leading to the dissociation and ionization of air molecules.

Plasma Dynamics and Communication Blackout

As detailed in the foundational research, the electron number density in the plasma sheath can exceed critical frequencies, blocking radio frequency (RF) communications (the "blackout" phenomenon). Modeling this requires resolving the transport of electrons and ions under the influence of self-induced electromagnetic fields. Traditional fluid models (Navier-Stokes) fail in the rarefied regions of the shock, necessitating kinetic approaches like DSMC or Particle-In-Cell (PIC) methods. However, these kinetic methods are computationally expensive due to the need to track billions of particles to minimize statistical noise.

The plasma sheath is not merely a fluid phenomenon but an electromagnetic one, where the permittivity of the medium changes dynamically with the electron density. This creates a non-linear feedback loop where the RF transmission properties are coupled to the aerothermodynamic state of the gas. The "blackout" occurs when the plasma frequency, ω_p, which is proportional to the square root of the electron number density (n_e), exceeds the frequency of the communication signal. Accurately predicting ω_p requires resolving the chemical kinetics of ionization, which are often in a state of thermal non-equilibrium. The electron temperature (T_e) can differ significantly from the heavy-particle translational temperature (T_{tr}) and the vibrational temperature (T_v), requiring a multi-temperature model that legacy codes often simplify or neglect.

Near-Space Weather Encounters

The original project scope highlights "Near-Space Weather." In the upper atmosphere (mesosphere/thermosphere), the vehicle is exposed not just to aerodynamic heating but also to solar energetic particles, cosmic rays, and micrometeoroids. These external radiation sources can enhance ionization in the shock layer, altering the chemical potential at the vehicle surface. A revamp of the code must, therefore, couple the internal gas dynamics with external radiation models, requiring a multi-physics solver capable of handling disparate time scales-from femtosecond electron plasma oscillations to millisecond aerodynamic flow adjustments.

Furthermore, "weather" in the mesosphere includes the presence of noctilucent clouds and charged dust particles. The impact of these particulates on the vehicle surface at hypersonic speeds (2000+ m/s) causes localized "spallation" and flash vaporization, injecting sudden bursts of high-density material into the boundary layer. Modeling these discrete events requires a Lagrangian tracking framework capable of handling millions of distinct particulate trajectories simultaneously with the continuum flow solver.

1.2 Material Response: Ablation and Surface Catalysis

The "Novel Material Systems" component of the research refers to thermal protection systems (TPS) designed to withstand heat fluxes via ablation-the process where surface material is removed through vaporization, sublimation, or erosion.

Multi-Phase Hydrodynamics

Ablation is inherently a multi-phase problem. As the material heats, it creates a "Knudsen layer" at the surface-a non-equilibrium region where continuum assumptions break down. The injection of ablation products (e.g., carbon species from a phenolic impregnator) into

the boundary layer alters the shock chemistry and thermodynamic state. The interaction is bi-directional: the flow heats the material, causing pyrolysis and gas injection, which in turn thickens the boundary layer and ("blowing effect") reduces the convective heat flux but potentially increases radiative heating due to the presence of carbon species like C_2 and CN.

Accurate simulation requires resolving the "blocking" effect of the ablation blowing, which creates a transpiration layer that separates the hot shock gas from the vehicle surface. This layer is turbulent, chemically reactive, and optically thick to radiation, requiring a tight coupling between the solid conduction solver, the fluid dynamics solver, and the radiative transfer equation.

Computational Bottleneck

Simulating ablation requires coupling a solid conduction solver (inside the material) with a flow solver (outside). Legacy codes often use loose coupling, exchanging boundary conditions at fixed intervals. This creates latency and stability issues. Project AETHER proposes a monolithic differentiable solver, where the solid and fluid domains are solved simultaneously on the GPU, allowing gradients to backpropagate through the interface for material property optimization.

The traditional "loose coupling" approach, often referred to as "Schwarz splitting," involves solving the fluid domain with a fixed wall temperature, calculating the heat flux, passing this flux to the solid solver, updating the wall temperature, and repeating. This iterative process is slow and often unstable for high-speed flows where the timescales of fluid advection and solid conduction differ by orders of magnitude. A monolithic approach, where the solid and fluid are treated as a single unified domain with spatially varying properties (permeability, conductivity), allows for implicit time integration that is unconditionally stable, drastically reducing the time-to-solution.

1.3 The Legacy Architecture vs. Modern Hardware

The prior work likely relied on MPI-based parallelization of CPU codes (e.g., implementing standard DSMC or Navier-Stokes solvers). While scalable on CPU clusters, these approaches are inefficient on modern GPU-centric supercomputers due to several fundamental architectural mismatches.

To fully utilize modern GPU architectures, one must refactor these algorithms for SIMT architectures, maximizing occupancy and memory throughput on architectures such as NVIDIA H100 or Blackwell. The transition involves not just a "port" of the code (e.g., using OpenACC directives) but a fundamental "rewrite" of the algorithms to be data-parallel. For instance, replacing the stochastic "accept-reject" sampling in DSMC (which causes divergence) with "inverse transform sampling" or pre-computed lookup tables that allow lock-step execution across a warp.

2 Part II: The Differentiable Physics Engine (JAX & Triton)

The core innovation of Project AETHER is the replacement of static C++/Fortran solvers with a JAX-based differentiable physics engine. JAX provides a NumPy-like interface with composable transformations for automatic differentiation (Autodiff), Just-In-Time (JIT) compi-

Bottleneck	Legacy CPU Approach	Modern GPU Requirement
Memory Access	Random access patterns (pointer chasing) in unstructured meshes are tolerated by large CPU caches.	GPUs require coalesced memory access; random access causes massive latency penalties and bandwidth waste.
Parallelism	Task-based parallelism (MPI) with heavy branching logic.	SIMT (Single Instruction, Multiple Threads) execution requires minimizing thread divergence. Branching logic (if/else) serializes warp execution.
Compute Density	Optimized for scalar processing and complex control flow.	GPUs are throughput-oriented devices optimized for massive vector/matrix operations. Legacy scalar loops utilize $< 1\%$ of peak FLOPs.
Data Movement	Frequent data exchange between solver modules (Fluid $\leftrightarrow$ Chemistry $\leftrightarrow$ Radiation) via main memory.	Data must remain in High Bandwidth Memory (HBM) or, ideally, SRAM (Shared Memory/L1) to avoid the ”memory wall”.

Table 1: Architectural Mismatches

lation (via XLA), and parallelization. This enables ”inverse design”-optimizing the material properties by differentiating the entire simulation with respect to the input parameters.

2.1 Transitioning to JAX: Just-In-Time Compilation for PDE Solvers

The project begins by porting the governing equations (Navier-Stokes for continuum, Boltzmann for rarefied) to JAX. The shift to JAX is strategic: it aligns the simulation codebase with the dominant ecosystem of deep learning, facilitating the integration of neural networks (surrogates) directly into the physics loop without overhead-heavy ”glue code” or interprocess communication.

XLA (Accelerated Linear Algebra) Optimization

Standard Python interpretation is too slow for HPC. JAX leverages XLA to compile computation graphs into optimized machine code. For the hypersonic flow solver, we utilize XLA's ability to fuse operations. In a finite volume scheme, the reconstruction (calculating values at cell faces), flux calculation (Riemann solver), and time integration (Runge-Kutta) steps involve reading and writing the same state vector multiple times. In a legacy code, these would be separate kernels, requiring multiple round-trips to HBM. XLA performs ”kernel fusion,” combining these steps into a single kernel that reads the state once, performs all arithmetic in registers, and writes the result once. This reduces the memory bandwidth requirement by a factor of 3-4x, which is the primary bottleneck for CFD solvers.

Differentiable DSMC and Gradient-Based Design

Implementing DSMC in JAX allows for differentiable particle tracking. By using the "reparameterization trick" (common in VAEs) or smooth approximations of discrete collisions, we can compute gradients of the drag and heat flux with respect to the surface geometry. This turns the simulation into a design tool. The challenge lies in the stochastic nature of DSMC. Project AETHER utilizes JAX's pseudo-random number generation (PRNG) keys, which are splittable and deterministic, ensuring reproducibility in parallel execution.

Standard DSMC involves discrete choices (e.g., "collide" or "don't collide"). These discontinuities break differentiation. We employ "Gumbel-Softmax" relaxation or "pathwise derivative" estimators to smooth these transitions, allowing gradients to flow through the stochastic sampling steps. This capability enables the automatic optimization of the TPS microstructure (e.g., pore size distribution) to minimize thermal conductivity, a task that would otherwise require thousands of forward simulations.

2.2 Custom Kernel Engineering with OpenAI Triton

While JAX/XLA provides high-level optimizations, specific operations in hypersonic simulation-such as particle-mesh interpolation and collision sorting-require low-level control to achieve peak performance. OpenAI Triton serves as the bridge, allowing the development of custom GPU kernels in Python that rival hand-tuned CUDA. This necessitates identifying when the compiler (XLA) is suboptimal and intervening with hand-written kernels.

Fused Kernels for Particle Operations

In a PIC or DSMC simulation, the "Move" and "Collide" phases are bottlenecks. The "Move" phase is memory-bound (streaming particle data), while the "Collide" phase is compute-bound but suffers from irregular memory access (particles in the same cell are not contiguous in memory).

- **The Problem:** Particles moving between cells require atomic operations to update grid moments (density, momentum), leading to memory contention. Standard `atomicAdd` operations on global memory are serialized and slow when many particles contend for the same cell (e.g., in high-density shock regions).
- **The Triton Solution:** We implement a block-based sorting kernel. Particles are first sorted by their cell index using a fast radix sort implemented in Triton. This ensures that threads in a warp access contiguous memory (coalescing), significantly increasing throughput. We utilize a "structure-of-arrays" (SoA) layout for particle data (x, y, z, u, v, w) to maximize cache line utilization.
- **Kernel Fusion:** We fuse the "particle push" (position update) and "boundary check" operations into a single Triton kernel. This keeps particle data in the GPU's SRAM (L1 cache/Shared Memory) avoids writing intermediate states to global HBM.

Snippet Insight 15: The snippet demonstrates a fused softmax kernel where data is kept in SRAM to maximize reuse. Project AETHER applies this logic to the Maxwell-Boltzmann

distribution sampling in DSMC. Instead of reading random numbers from global memory (which consumes bandwidth), we generate them on-the-fly in registers using counter-based random number generators (CBRNG) like Philox or Threefry, and compute the post-collision velocities within the same kernel launch. This "compute-heavy, memory-light" approach is optimal for modern GPUs where FLOPs are cheap but bandwidth is expensive.

Optimizing Memory Access in Particle-In-Cell (PIC)

For the plasma sheath simulation, the project utilizes a Triton-based PIC implementation. The electromagnetic field interpolation (gathering field values from grid to particle) is bandwidth-bound.

- **Tiling Strategy:** We use Triton's block pointers to load 2D/3D tiles of the electromagnetic field into shared memory. The particles processing in that block then query the shared memory cache rather than global memory. This effectively explicitly manages the cache, preventing "cache thrashing" when particles cross block boundaries.
- **Autotuning:** Triton's `@triton.autotune` decorator is used to automatically find the optimal tile sizes (e.g., 128×128 vs 64×256) and the number of warps for different GPU architectures (e.g., A100 vs H100). This ensures portability and performance sustainability.

2.3 Low-Level Hardware Control: Pallas & Mosaic

To maximize hardware utilization and fine-grained control, Project AETHER pushes beyond Triton into JAX Pallas, a kernel language that exposes the specific memory hierarchies of TPUs and GPUs. This shows the ability to program "bare metal" within a high-level framework.

Mosaic-GPU for Tensor Core Utilization

Hypersonic simulations involve heavy matrix multiplications (e.g., in implicit time integration schemes or chemical kinetics solvers). JAX Pallas allows us to program the Tensor Memory Accelerator (TMA) on NVIDIA Hopper GPUs explicitly.

- **Pipeline Management:** We implement a "software pipeline" using Pallas. While the Tensor Cores compute the reaction rates for batch N, the TMA asynchronously loads the state vectors for batch $N+1$ from global memory to shared memory. This overlaps compute and data transfer, hiding the latency of HBM access.
- **Barrier Synchronization:** Pallas provides explicit control over barriers (`mbarrier` on CUDA). We use this to synchronize threads only when necessary (e.g., after writing to shared memory), avoiding the global synchronization penalties of standard `_syncthreads()` in high-divergence flows. This fine-grained synchronization is essential for "warp-specialized" kernels where different warps in a block perform different tasks (e.g., one warp loads data, three warps compute).

3 Part III: Scientific Machine Learning & Surrogate Architectures

The simulation of hypersonic flight over a full trajectory (minutes to hours) using full-fidelity DSMC/CFD is computationally prohibitive. A single high-fidelity simulation point can take thousands of GPU hours. To enable "Near-Space Weather" prediction in real-time (for flight control), Project AETHER incorporates Scientific Machine Learning (SciML) to create surrogate models that accelerate the simulation by orders of magnitude while preserving physical accuracy.

3.1 Physics-Informed Transformers (PITs)

Transformers have revolutionized natural language processing (NLP), but their application to physics is nascent and high-value. We propose a Physics-Informed Transformer (PIT) architecture to predict the temporal evolution of the flow field.

Architecture: Graph-to-Sequence Modeling

- **Tokenizer:** The flow field (density, pressure, temperature) on the unstructured mesh is tokenized using a Graph Neural Network (GNN) encoder. Each mesh node or particle cluster becomes a token. We use a Message Passing Neural Network (MPNN) to aggregate local neighborhood information (gradients, fluxes) into the node embedding before passing it to the Transformer. This ensures the token representation respects the local topology of the mesh.
- **Attention Mechanism:** A self-attention mechanism captures the long-range dependencies in the flow (e.g., how a shock wave at the nose affects the wake). Standard attention scales quadratically $O(N^2)$, which is too slow for millions of mesh points.
- **Linear Attention & Galerkin Transformers:** We employ linear complexity attention mechanisms (like Galerkin attention) or factorized attention (FactFormer) to handle the high dimensionality of CFD data. The "Factorized Transformer" decomposes the 3D attention integral into three 1D integrals along the x, y, and z axes, reducing complexity from $O(N^2)$ to $O(N)$.

Physics-Informed Loss

Unlike standard LLMs trained on text, the PIT is trained to minimize the residual of the Navier-Stokes equations (physics loss) alongside data error.

$$\mathcal{L}_{total} = \mathcal{L}_{data} + \lambda \cdot ||\mathcal{N}(u_{pred})||^2$$

Where $\mathcal{N}$ is the non-linear differential operator defining the fluid laws (Conservation of Mass, Momentum, Energy). This ensures that the predicted flow fields conserve physical quantities, preventing the "hallucinations" common in pure data-driven models. This technique, known as Physics-Informed Neural Networks (PINNs), is adapted here for the Transformer architecture, allowing the model to generalize to flow regimes outside its training data (e.g., higher Mach numbers) by adhering to the underlying physics.

3.2 Star Attention for Long-Sequence Trajectories

Simulating the "weather encounter" implies long-duration tracking of particle interactions. The context window of standard Transformers is a limitation. Project AETHER adapts Star Attention, a technique developed for long-context LLMs, to particle physics.

Block-Sparse Physics Attention

- **Concept:** Star Attention divides the sequence (time steps of particle trajectory) into local blocks. A "global" token summarizes the state of the system (e.g., total energy, global shock position).
- **Implementation:** Local attention processes the immediate history of a particle (micro-physics), while global attention allows the particle to attend to the macroscopic state of the flow (macro-physics). This reduces inference time by up to 11x compared to dense attention.
- **Distributed Inference:** We shard the long trajectory data across multiple GPUs. The Star Attention mechanism minimizes communication by only exchanging the global tokens between hosts, enabling the simulation of extremely long flight times without memory overflow. This mirrors the "Ring Attention" or "Sequence Parallelism" techniques used in training Large Language Models, demonstrating the application of GenAI infrastructure to scientific problems.

3.3 Transformer-Based Particle Tracking (MOT)

In the rarefied regime, tracking individual dust or ice particles (the "weather" elements) colliding with the vehicle is a Multiple Object Tracking (MOT) problem.

TrackFormer for Lagrange Particles

We adapt the TrackFormer or MOTR (Motion-Aware Transformer) architectures.

- **Track Queries:** Each physical particle is represented by a learnable "track query" vector. The Transformer decoder updates these queries frame-by-frame based on the forces (drag, lift, electrostatic) predicted by the flow model.
- **End-to-End Association:** Traditional algorithms use Kalman filters and Hungarian matching (bipartite graph matching) to associate particles between frames. These algorithms are $O(N^3)$ and hard to parallelize. The Transformer handles association and state update in a single parallelizable pass, implicitly learning the motion model. This allows tracking millions of particles simultaneously on the GPU, a capability required for realistic "weather" simulation where particle counts can be massive.

4 Part IV: High-Performance Numerics & Quantization

To maximize "performance per watt" and "performance per dollar", Project AETHER aggressively employs reduced precision arithmetic and sparse linear algebra. This moves be-

yond the standard ”FP64 for everything” mentality of academic CFD, adopting the ”mixed precision” philosophy of Deep Learning.

4.1 Mixed-Precision Aerothermodynamics: FP8 & INT8

Standard CFD uses double precision (FP64) to avoid accumulation of round-off errors. However, deep learning surrogates and even parts of the explicit time integration can tolerate lower precision if managed correctly.

FP8 for Surrogate Inference

The inference pass of the Physics-Informed Transformer is memory-bandwidth bound. By quantizing the weights and activations to FP8 (E4M3 or E5M2), we reduce the memory footprint by 4x compared to FP32 and 2x compared to FP16.

- **Implementation:** We use the NVIDIA Transformer Engine to cast the linear layers of the PIT to FP8. This utilizes the fourth-generation Tensor Cores on H100 GPUs, doubling the throughput.
- **Residual Correction:** To maintain physical accuracy, we use a mixed-precision approach: the bulk flow prediction is done in FP8, but a ”residual correction” step (calculating the error against the conservation laws) is performed in FP32. This recovers the precision lost during quantization. This ”iterative refinement” allows us to achieve FP32 accuracy with FP8 storage costs.

INT8 for Particle Sorting

For the sorting and binning of particles in DSMC, precision is less critical than integer throughput. We utilize INT8 tensor operations for the radix sort keys, leveraging the specialized INT8 processing units on modern GPUs to accelerate the neighbor search. The cell indices and particle IDs are mapped to integers, and the sorting network (a bitonic sort or radix sort) is implemented using INT8 vector instructions (dp4a), which are significantly faster than FP32 equivalents.

4.2 Sparse Matrix Operations with CUTLASS

Reacting hypersonic flows involve complex chemical kinetics (e.g., 5-species or 11-species air models). The Jacobian matrix describing the reaction rates is large but sparse (species only react with specific other species).

Block-Sparse GEMM

Standard dense matrix multiplication (GEMM) wastes compute on zeros. Project AETHER utilizes CUTLASS (CUDA Templates for Linear Algebra Subroutines) to implement Block-Sparse Matrix Multiplication.

- **Structured Sparsity:** We reorder the species vector to cluster interacting species, creating dense blocks within the sparse Jacobian. This is known as ”Reverse Cuthill-McKee” ordering or similar graph permutation techniques to bandwidth-minimize the matrix.
- **Direct Register Access:** CUTLASS allows us to define C++ templates that load these blocks directly into the GPU registers. This bypasses the cache hierarchy for the inner loop of the kinetics solver, resulting in a 2-3x speedup over standard cuBLAS for chemical kinetics. This takes full advantage of the GPU’s register file and instruction scheduling for maximum performance.

5 Part V: Distributed Computing & Infrastructure

Scaling Project AETHER to ”Exascale” (simulating a full vehicle with detailed microstructure) requires distributed computing across thousands of GPUs. This requires advanced systems engineering typically used in large-scale AI labs, shifting focus to robust backend infrastructure.

5.1 Disaggregated Serving for Digital Twins

In a real-world scenario, the ”Near-Space Weather” system acts as a digital twin, receiving sensor data from the flight and predicting future states. We propose a Disaggregated Serving architecture, commonly used for LLMs, to handle this.

Prefill vs. Decode in Physics

- **Prefill Cluster (The Simulator):** A set of high-compute GPUs (e.g., H100s) runs the heavy JAX/Triton physics engine to generate the initial state and ”fill” the context window of the Transformer surrogate. This is compute-bound, requiring massive FLOPS for the initial convergence.
- **Decode Cluster (The Predictor):** A separate set of high-memory bandwidth GPUs (e.g., A100s or L40s) runs the quantized (FP8) Transformer surrogate to autoregressively predict the future flow state token-by-token. This is memory-bound.
- **KV Cache Management:** We utilize PagedAttention (from vLLM) to manage the memory of the ”physics state cache” (analogous to the KV cache in LLMs). In a simulation, the ”state” history grows over time. PagedAttention allows non-contiguous memory allocation for the time-history of different mesh regions, eliminating fragmentation and allowing dynamic mesh refinement (AMR) without reallocating the entire buffer. This is a direct translation of LLM serving optimization to computational physics.

5.2 Custom Interconnects: NCCL & SCCL

For the distributed training of the surrogate models and the multi-GPU DSMC simulation, standard MPI is insufficient. We utilize NCCL (NVIDIA Collective Communication Library) for topology-aware communication.

Synthesized Collectives (SCCL)

Hypersonic meshes are often unstructured and load-imbalanced (more particles at the shock wave). A standard AllReduce is inefficient because it assumes uniform data distribution. We use SCCL (Synthesized Collective Communication Library) to generate custom collective algorithms tailored to the specific mesh topology and the physical wiring of the GPU cluster (e.g., NVLink vs. InfiniBand).

- **Ring vs. Tree:** For the shock layer regions (dense data), SCCL synthesizes a bandwidth-optimal Ring algorithm. For the rarefied wake (sparse data), it switches to a latency-optimal Tree algorithm. This dynamic switching based on flow regime is a novel application of SCCL, demonstrating deep understanding of distributed system bottlenecks.

5.3 Sharding & Domain Decomposition

To handle the massive scale, we employ 4D parallelism, inspired by techniques for training trillion-parameter models:

1. **Data Parallelism:** Different ensemble members (weather scenarios) run on different GPU groups.
2. **Tensor Parallelism:** The large weight matrices of the surrogate model are split across GPUs within a node (using NVLink).
3. **Pipeline Parallelism:** The layers of the physics solver (e.g., Advection $\rightarrow$ Diffusion $\rightarrow$ Chemistry) are pipelined across nodes.
4. **Domain/Sequence Parallelism:** The spatial mesh is decomposed using a Hilbert curve to preserve locality.

We use JAX's `pmap` and `shard_map` to explicitly define how the mesh shards map to the TPU/GPU mesh. This explicit control over data layout is superior to the "black box" parallelism of standard MPI solvers, allowing for perfect load balancing even in highly dynamic flows.

6 Part VI: Strategic Implementation Roadmap

1. **Phase I: The Kernel Rewrite (Months 0-6):**
    - Replace legacy Fortran routines with JAX.
    - Develop Triton kernels for the DSMC collision integral.
    - **Deliverable:** A single-GPU benchmark showing 50x speedup over CPU baseline.
2. **Phase II: The Surrogate Training (Months 6-12):**
    - Generate training data using the Phase I solver.

- Train the Physics-Informed Transformer (PIT) using FP8 quantization on a distributed cluster.
- **Deliverable:** A surrogate model capable of real-time flow inference with $< 1\%$ error.

3. **Phase III: The Exascale Integration (Months 12-18):**
    - Deploy the Disaggregated Serving architecture.
    - Implement Star Attention for long-duration flight simulation.
    - **Deliverable:** A "Digital Twin" demonstration tracking millions of particles in near-real-time.

Conclusion

Project AETHER represents a paradigm shift from empirical, computationally expensive validation to predictive, AI-accelerated design. By fusing the rigorous physics of hypersonic aerothermodynamics with the explosive performance of modern AI hardware and architectures, this project not only solves the specific problem of "Near-Space Weather Encounters" but also serves as a definitive proof-of-capability for next-generation aerospace simulations. It establishes a robust foundation for computational science, capable of bridging the gap between theoretical physics and exascale systems engineering. This holistic approach is essential for advancing the modern AI and HPC landscape.

Works cited

1. Director Kommalapati - C.E.E.S. - Prairie View A&M University. https://www.pvamu.edu/cees/mission/kommalapati/
2. Research - Plasma Dynamics Modeling Laboratory. https://pdml.stanford.edu/research
3. Investigating Hypersonic Plasma Sheath and Heat Ablation Effects on RF Transmission using DSMC Method - Washington University in St. Louis Scholarly Repository. https://openscholarship.wustl.edu/mems500/154/
4. A STUDY INTO VALIDATING A COUPLED METHOD OF CHARACTERISTICS AND DIRECT SIMULATION MONTE CARLO METHOD AGAINST EMPIRICAL DATA by A - NASA Technical Reports Server. https://ntrs.nasa.gov/api/citations/20205004477/downloads/Thesis_ITAR_SubmissionCk.pdf
5. Direct simulation Monte Carlo investigation of the Rayleigh-Taylor instability - ResearchGate. https://www.researchgate.net/publication/307552586_Direct_simulation_Monte_Carlo_investigation_of_the_Rayleigh-Taylor_instability
6. Investigating Hypersonic Plasma Sheath and Heat Ablation Effects on RF Transmission using DSMC Method - Washington University in St. Louis Scholarly Repository.

https://openscholarship.wustl.edu/cgi/viewcontent.cgi?article=1165&context=mems500

7. Laboratory simulation of laser propagation through plasma sheaths containing ablation particles of ZrB2-SiC-C during hypersonic flight | Request PDF - ResearchGate. https://www.researchgate.net/publication/313367082_Laboratory_simulation_of_laser_propagation_through_plasma_sheaths_containing_ablation_particles_of_ZrB2-SiC-C_during_hypersonic_flight
8. Computational Models for Nanosecond Laser Ablation - Scholarly Commons. https://commons.erau.edu/cgi/viewcontent.cgi?article=1900&context=publication
9. GPU Acceleration of an Adjoint Enabled Real Gas ... - NASA TechPort. https://techport.nasa.gov/projects/113237
10. The Influence of Computer Architecture on Performance and Scaling for Hypersonic Flow Simulations - NASA Technical Reports Server (NTRS). https://ntrs.nasa.gov/citations/20205011394
11. Navier-Stokes and DSMC simulations for hypersonic laminar shock-shock interaction flows. https://www.semanticscholar.org/paper/8612ec27271d53de66d47e6b949a7ae294bd6c93
12. (PDF) A Multi-GPU Parallel Algorithm in Hypersonic Flow ... https://www.researchgate.net/publication/331840886_A_Multi-GPU_Parallel_Algorithm_in_Hypersonic_Flow_Computations
13. Triton Kernel Implementation - Emergent Mind. https://www.emergentmind.com/topics/triton-kernel-implementation
14. Advancing GPU Programming with the CUDA Tile IR Backend for OpenAI Triton. https://developer.nvidia.com/blog/advancing-gpu-programming-with-the-cuda-tile-ir-backend-for-openai-triton/
15. Introducing Triton: Open-source GPU programming for neural networks - OpenAI. https://openai.com/index/triton/
16. Pallas Quickstart - JAX documentation. https://docs.jax.dev/en/latest/pallas/quickstart.html
17. Quickstart: How to think in JAX - JAX documentation. https://docs.jax.dev/en/latest/quickstart.html
18. jax-ml/jax: Composable transformations of Python+NumPy programs. https://github.com/jax-ml/jax
19. JAX-in-Cell: A Differentiable Particle-in-Cell Code for Plasma Physics Applications | Cool Papers. https://papers.cool/arxiv/2512.12160
20. A shared compilation stack for distributed-memory parallelism in stencil DSLs | Request PDF. https://www.researchgate.net/publication/380150959_A_shared_compilation_stack_for_distributed-memory_parallelism_in_stencil_DSLs

21. Domain-Specific Compilation Framework with High-Level Tensor Abstraction for Fast Fourier Transform and Finite-Difference Time-D - Diva-portal.org. https://www.diva-portal.org/smash/get/diva2:1958857/FULLTEXT01.pdf

22. Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations. https://www.eecs.harvard.edu/~htk/publication/2019-mapl-tillet-kung-cox.pdf

23. Kernel development and optimization with Triton - AMD ROCm documentation. https://rocm.docs.amd.com/projects/ai-developer-hub/en/latest/notebooks/gpu_dev_optimize/triton_kernel_dev.html

24. Learning Triton One Kernel At a Time: Vector Addition - Towards Data Science. https://towardsdatascience.com/learning-triton-one-kernel-at-a-time-vector-addition/

25. Fused Softmax - Triton documentation. https://triton-lang.org/main/getting-started/tutorials/02-fused-softmax.html

26. OpenAI's Triton: An end to end example | by Michael Diggin | Medium. https://medium.com/@michael.diggin/openais-triton-an-end-to-end-example-c6577d81e3d0

27. Triton GPU Kernel Programming Guide: CUDA Alternative - RightNow AI. https://www.rightnowai.co/guides/frameworks/triton

28. Matrix Multiplication - JAX documentation. https://docs.jax.dev/en/latest/pallas/tpu/matmul.html

29. Modular GPU Programming with Typed Perspectives - arXiv. https://arxiv.org/pdf/2511.11939

30. (PDF) Physics-Informed Transformers - ResearchGate. https://www.researchgate.net/publication/398054286_Physics-Informed_Transformers

31. Scalable Transformer for PDE Surrogate Modeling - NeurIPS. https://proceedings.neurips.cc/paper_files/paper/2023/file/590daf74f99ee85df3d8c007df9c8187-Paper-Conference.pdf

32. Physics-Conditioned Transformer for Surrogate Modelling of Aerofoil Pressure Distribution - The University of Liverpool Repository. https://livrepository.liverpool.ac.uk/3196549/1/ANHICHEM_AIAA_Scitech_Forum_2026_Manuscript.pdf

33. Physics-Informed Surrogate Modelling in Fire Safety Engineering: A Systematic Review. https://www.mdpi.com/2076-3417/15/15/8740

34. Physics-Informed Mixture Models and Surrogate Models for Precision Additive Manufacturing - arXiv. https://arxiv.org/html/2510.26586v1

35. Star Attention: Efficient LLM Inference over Long Sequences - OpenReview. https://openreview.net/forum?id=QY7Au9nZwp-eld=MozeEUBYdc

36. Preformer MOT: A transformer-based approach for multi-object tracking with global trajectory prediction - PMC. https://pmc.ncbi.nlm.nih.gov/articles/PMC12869739/

37. Motion-Aware Transformer for Multi-Object Tracking - arXiv. https://arxiv.org/html/2509.21715v1

2105.03247 MOTR: End-to-End Multiple-Object Tracking with Transformer - arXiv. https://arxiv.org/abs/2105.03247

38. FP8, BF16, and INT8: How Low-Precision Formats Are Revolutionizing Deep Learning Throughput | by StackGpu | Medium. https://medium.com/@StackGpu/fp8-bf16-and-int8-how-low-precision-formats-are-revolutionizing-deep-learning-throughput-e6c1f3adabc2

39. FP8 Quantization in Deep Neural Networks - Emergent Mind. https://www.emergentmind.com/topics/fp8-quantization

40. FP8 versus INT8 for efficient deep learning inference - arXiv. https://arxiv.org/pdf/2303.17951

41. What Is int8 Quantization and Why Is It Popular for Deep Neural Networks? - MathWorks. https://www.mathworks.com/company/technical-articles/what-is-int8-quantization-and-why-is-it-popular-for-deep-neural-networks.html

2303.17951 FP8 versus INT8 for efficient deep learning inference - arXiv. https://arxiv.org/abs/2303.17951

42. A method for accelerating low precision operations by sparse matrix multiplication - arXiv. https://arxiv.org/html/2403.06924v1

43. Accelerating Matrix Multiplication with Block Sparse Format and NVIDIA Tensor Cores. https://developer.nvidia.com/blog/accelerating-matrix-multiplication-with-block-sparse-format-and-nvidia-tensor-cores/

44. Implementing High Performance Matrix Multiplication Using CUTLASS v2.8. https://developer.nvidia.com/blog/implementing-high-performance-matrix-multiplication-using-cutlass-v2-8/

45. Disaggregation in Large Language Models: the Next Evolution in AI Infrastructure - InfoQ. https://www.infoq.com/articles/llms-evolution-ai-infrastructure/

46. Disaggregated inference - Modular Docs. https://docs.modular.com/mammoth/disaggregated-inference/

47. Disaggregated Inference: 18 Months Later | Hao AI Lab @ UCSD. https://hao-ai-lab.github.io/blogs/distserve-retro/

48. Revisiting Disaggregated Large Language Model Serving for Performance and Energy Implications - arXiv. https://arxiv.org/html/2601.08833v1

49. Fast Multi-GPU collectives with NCCL | NVIDIA Technical Blog. https://developer.nvidia.com/blog/fast-multi-gpu-collectives-nccl/

50. New Scaling Algorithm and Initialization with NVIDIA Collective Communications Library 2.23. https://developer.nvidia.com/blog/new-scaling-algorithm-and-initialization-with-nvidia-collective-communications-library-2-23/

51. Synthesizing optimal collective communication algorithms - Microsoft Research. https://www.microsoft.com/en-us/research/publication/synthesizing-optimal-collective-communication-algorithms/

52. Visualizing All-Reduce Bandwidth: The Physics of Distributed Training - Rajat Pandit. https://rajatpandit.com/visualizing-all-reduce-bandwidth

53. MLSys Poster Efficient Post-training Quantization with FP8 Formats. https://mlsys.org/virtual/2024/poster/2640